

Data Exploration

Introduction: For this assignment, I analyzed a corpus of Early Modern political economy texts provided on Canvas. These documents were intentionally selected as part of a shared intellectual tradition, allowing for systematic comparison. After cleaning and normalizing the texts, I conducted an exploratory analysis using three approaches: TF-IDF to identify lexically distinctive terms, Pearson correlation to measure similarity between documents, and syntactic complexity measures to compare structural writing patterns across two selected texts. Together, these methods allowed me to examine how the texts relate to one another both in vocabulary use and grammatical structure.

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.6
## ✓ forcats    1.0.1      ✓ stringr    1.6.0
## ✓ ggplot2    4.0.1      ✓ tibble     3.3.1
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.2
## ✓ purrr      1.2.1
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidytext)
library(stringr)
library(widyr)
```

```
getwd()
```

```
## [1] "/Users/sejal/Documents/Text_As_Data/text_as_data"
```

```
list.files("texts", full.names = TRUE)
```

```
## [1] "texts/A06785.txt" "texts/A06786.txt" "texts/A06788.txt" "texts/A06789.txt"
## [5] "texts/A06790.txt" "texts/A06791.txt" "texts/A07594.txt" "texts/A07886.txt"
## [9] "texts/A32827.txt" "texts/A32828.txt" "texts/A32829.txt" "texts/A32830.txt"
## [13] "texts/A32833.txt" "texts/A32836.txt" "texts/A32837.txt" "texts/A32838.txt"
## [17] "texts/A32839.txt" "texts/A50763.txt" "texts/A51598.txt" "texts/A69858.txt"
## [21] "texts/A93819.txt" "texts/B14801.txt" "texts/wealth.txt"
```

```
# read_file("texts/A06785.txt")
```

```
files <- list.files("texts",
                    pattern = "\\\\.txt$",
                    full.names = TRUE)

corpus_raw <- tibble(
  doc_id = tools::file_path_sans_ext(basename(files)),
  text = map_chr(files, read_file)
)

print(corpus_raw)
```

```
## # A tibble: 23 × 2
##   doc_id text
##   <chr> <chr>
## 1 A06785 "Strong imagination, nourished by opinion, may cause most men to supp...
## 2 A06786 "When Almighty God had created man, good and a sociable creature, who...
## 3 A06788 "A SENTENCE alleged without application to some purpose, is to handle...
## 4 A06789 "NATURAL Mother wit, did teach man, before Arts or Sciences were inve...
## 5 A06790 "APRIL having with his sweet showers moistened the drought of March, ...
## 6 A06791 "Plato the Philosopher perceiving that equality would be the cause th...
## 7 A07594 "THE CIRCLE OF COMMERCE. The Prooeme. HERODOTVS in his CLIO, reportes...
## 8 A07886 "The trade of Merchandise, is not only that laudable practice whereby...
## 9 A32827 "THE Prodigious increase of the Netherlanders in their Domestic and F...
## 10 A32828 "WHEN fallen Man was driven out of Paradise, the heavy Charge imposed...
## # i 13 more rows
```

```
# Count words that contain "f" before replacement
sum(str_detect(corpus_raw$text, "f"))
```

```
## [1] 1
```

Step 0: Normalization

In preparing the corpus for analysis, I adopted a minimal-intervention normalization strategy. The goal was to ensure lexical comparability across documents while preserving historically meaningful orthographic variation.

0.1) First, I replaced the Early Modern long S character (f) with the modern letter “s,” as required. Without this correction, words like “ftate” and “state” would be counted as different words, which would distort the analysis.

0.2) Second, I converted all text to lowercase and removed extra spaces. Lowercasing ensures that words such as “State” and “state” are treated as the same word. Collapsing extra whitespace prevents formatting inconsistencies from interfering with tokenization.

I chose not to modernize spelling or standardize variations such as u/v or archaic spellings. While doing so might make the texts look more uniform, it could also erase meaningful stylistic or historical differences between authors. Since this assignment focuses on exploring patterns rather than rewriting the texts, I aimed to preserve the original language as much as possible.

```
corpus_normalized <- corpus_raw %>%
  mutate(
    text = str_replace_all(text, "f", "s") %>%      # normalize long S
           str_replace_all("\\s+", " ") %>%         # collapse extra whitespace
           str_to_lower()                            # standardize case
  )

corpus_normalized
```

```
## # A tibble: 23 × 2
##   doc_id text
##   <chr>  <chr>
## 1 A06785 "strong imagination, nourished by opinion, may cause most men to supp...
## 2 A06786 "when almighty god had created man, good and a sociable creature, who...
## 3 A06788 "a sentence alleged without application to some purpose, is to handle...
## 4 A06789 "natural mother wit, did teach man, before arts or sciences were inve...
## 5 A06790 "april having with his sweet showers moistened the drought of march, ...
## 6 A06791 "plato the philosopher perceiving that equality would be the cause th...
## 7 A07594 "the circle of commerce. the prooeme. herodotvs in his clio, reportes...
## 8 A07886 "the trade of merchandise, is not only that laudable practice whereby...
## 9 A32827 "the prodigious increase of the netherlanders in their domestic and f...
## 10 A32828 "when fallen man was driven out of paradise, the heavy charge imposed...
## # i 13 more rows
```

During tokenization, I also removed punctuation, standalone numbers, common stopwords (like “the” and “and”), and tokens containing obvious OCR errors (such as • symbols). These elements do not carry meaningful content for this type of analysis and would otherwise introduce noise.

Overall, I used a minimal cleaning approach: correcting only what was necessary for accurate comparison while preserving the integrity of the original texts.

```
tokens_df <- corpus_normalized %>%
  unnest_tokens(word, text)
```

```
tokens_df <- tokens_df %>%
  anti_join(stop_words, by = "word") %>%      # remove stopwords
  filter(!str_detect(word, "^[0-9]+$")) %>%    # remove numbers
  filter(!str_detect(word, "●"))              # remove OCR artifacts

# tokens_df
```

```
#Check document length balance
tokens_df %>%
  count(doc_id, sort = TRUE)
```

```
## # A tibble: 23 × 2
##   doc_id      n
##   <chr>   <int>
## 1 wealth 128044
## 2 A06786  82055
## 3 A32833  16132
## 4 A32827  16098
## 5 A07594  13311
## 6 A06785  12894
## 7 A06788   9968
## 8 A51598   9211
## 9 A06789   8587
## 10 B14801   8139
## # i 13 more rows
```

```
# Check most frequent words
```

```
tokens_df %>%
  count(word, sort = TRUE) %>%
  slice_max(n, n = 20)
```

```
## # A tibble: 21 × 2
##   word      n
##   <chr>    <int>
## 1 trade    3234
## 2 money    3224
## 3 price    2093
## 4 commodities 1967
## 5 time     1866
## 6 country   1684
## 7 exchange  1641
## 8 silver    1470
## 9 merchants 1391
## 10 people   1306
## # i 11 more rows
```

```
# Vocabulary size per document
```

```
tokens_df %>%
  group_by(doc_id) %>%
  summarise(unique_words = n_distinct(word))
```

```
## # A tibble: 23 × 2
##   doc_id unique_words
##   <chr>    <int>
## 1 A06785    3639
## 2 A06786   11193
## 3 A06788    2774
## 4 A06789    2707
## 5 A06790    2234
## 6 A06791    1556
## 7 A07594    4234
## 8 A07886    1534
## 9 A32827    3886
## 10 A32828   1053
## # i 13 more rows
```

```
# Check for any non-alphabetic tokens
```

```
tokens_df %>%
  filter(str_detect(word, "[^a-z]")) %>%
  count(word, sort = TRUE)
```

```
## # A tibble: 1,019 × 2
##   word          n
##   <chr>      <int>
## 1 merchant's   228
## 2 king's       149
## 3 majesty's    149
## 4 men's        117
## 5 man's        114
## 6 butcher's     54
## 7 country's     40
## 8 england's     35
## 9 prince's      35
## 10 god's        33
## # i 1,009 more rows
```

These are real words and carry meaning, not OCR errors.

APPROACH 1: TF - IDF I computed TF-IDF scores for each document to identify terms that are distinctive relative to the rest of the corpus. This allowed me to see what makes each text lexically unique while still operating within a shared vocabulary.

```
# Step 1: Document feature matrix

dfm_counts <- tokens_df %>%
  count(doc_id, word, sort = TRUE)

dfm_counts
```

```
## # A tibble: 60,375 × 3
##   doc_id word          n
##   <chr> <chr>      <int>
## 1 wealth price      1264
## 2 wealth country    1240
## 3 wealth labor      1011
## 4 wealth trade       970
## 5 wealth produce     945
## 6 wealth quantity    797
## 7 wealth people      777
## 8 wealth money       770
## 9 A06786 money       750
## 10 wealth land       720
## # i 60,365 more rows
```

```
# Step 2: Compute TF-IDF weights
```

```
tf_idf_data <- dfm_counts %>%
  bind_tf_idf(word, doc_id, n) %>%
  arrange(desc(tf_idf))
```

```
tf_idf_data
```

```
## # A tibble: 60,375 × 6
##   doc_id word      n      tf   idf tf_idf
##   <chr> <chr>   <int>  <dbl> <dbl> <dbl>
## 1 A93819 key      29 0.0294  1.75 0.0513
## 2 A93819 stairs   23 0.0233  1.75 0.0407
## 3 A07594 haue    163 0.0122  2.44 0.0299
## 4 A93819 lion     17 0.0172  1.53 0.0263
## 5 B14801 haue     86 0.0106  2.44 0.0258
## 6 A32838 surrat   11 0.0102  2.44 0.0250
## 7 A07594 malynes 158 0.0119  2.04 0.0242
## 8 B14801 kingdome 75 0.00921 2.44 0.0225
## 9 A07594 hee     115 0.00864 2.44 0.0211
## 10 A07594 kingdome 113 0.00849 2.44 0.0207
## # i 60,365 more rows
```

```
# Step 3: Top distinctive terms per document
```

```
top_terms <- tf_idf_data %>%
  group_by(doc_id) %>%
  slice_max(tf_idf, n = 15) %>%
  ungroup()
```

```
top_terms
```

```
## # A tibble: 376 × 6
##   doc_id word          n      tf   idf   tf_idf
##   <chr>  <chr>      <int>   <dbl> <dbl>   <dbl>
## 1 A06785 moneys      264 0.0205  0.651 0.0133
## 2 A06785 misselden    36 0.00279 3.14  0.00875
## 3 A06785 starlin     33 0.00256 2.44  0.00625
## 4 A06785 exchange   288 0.0223  0.245 0.00548
## 5 A06785 circle     41 0.00318 1.53  0.00485
## 6 A06785 realm      64 0.00496 0.938 0.00466
## 7 A06785 doller     28 0.00217 2.04  0.00442
## 8 A06785 undervaluation 36 0.00279 1.53  0.00426
## 9 A06785 royals     26 0.00202 2.04  0.00411
## 10 A06785 stivers    37 0.00287 1.34  0.00386
## # i 366 more rows
```

```
top_terms <- tf_idf_data %>%
  group_by(doc_id) %>%
  slice_max(tf_idf, n = 15) %>%
  ungroup()
```

```
colnames(top_terms)
```

```
## [1] "doc_id" "word"    "n"      "tf"     "idf"    "tf_idf"
```

```
# Export one TF-IDF table per document

# Split TF-IDF results by document
doc_list <- split(top_terms, top_terms$doc_id)

dir.create("tfidf_results", showWarnings = FALSE)

purrr::walk(names(doc_list), function(doc) {
  write_csv(doc_list[[doc]],
            file.path("tfidf_results",
                      paste0("tf_idf_", doc, ".csv")))
})
```



```
# Create folder for TF-IDF distinctiveness results

dir.create("tfidf_distinctiveness", showWarnings = FALSE)

# Split results by document
doc_list <- split(top_terms, top_terms$doc_id)

# Export each document's distinctive terms
purrr::walk(names(doc_list), function(doc) {
  write_csv(doc_list[[doc]],
            file.path("tfidf_distinctiveness",
                      paste0("distinctive_terms_", doc, ".csv")))
})
```

```
list.files("tfidf_distinctiveness")
```

```
## [1] "distinctive_terms_A06785.csv" "distinctive_terms_A06786.csv"
## [3] "distinctive_terms_A06788.csv" "distinctive_terms_A06789.csv"
## [5] "distinctive_terms_A06790.csv" "distinctive_terms_A06791.csv"
## [7] "distinctive_terms_A07594.csv" "distinctive_terms_A07886.csv"
## [9] "distinctive_terms_A32827.csv" "distinctive_terms_A32828.csv"
## [11] "distinctive_terms_A32829.csv" "distinctive_terms_A32830.csv"
## [13] "distinctive_terms_A32833.csv" "distinctive_terms_A32836.csv"
## [15] "distinctive_terms_A32837.csv" "distinctive_terms_A32838.csv"
## [17] "distinctive_terms_A32839.csv" "distinctive_terms_A50763.csv"
## [19] "distinctive_terms_A51598.csv" "distinctive_terms_A69858.csv"
## [21] "distinctive_terms_A93819.csv" "distinctive_terms_B14801.csv"
## [23] "distinctive_terms_wealth.csv"
```

APPROACH 2 - Pearson Correlation Using a document-feature matrix, I calculated pairwise Pearson correlations between documents to measure similarity in vocabulary distributions. I visualized these relationships with a heatmap and identified the most and least similar document pairs.

```
# Load required libraries
```

```
library(quanteda)
```

```
## Package version: 4.3.1
```

```
## Unicode version: 14.0
```

```
## ICU version: 71.1
```

```
## Parallel computing: disabled
```

```
## See https://quanteda.io for tutorials and examples.
```

```
library(quanteda.textstats)  
library(tidyverse)
```

```
# Create quanteda corpus
```

```
corpus_quanteda <- corpus(corpus_normalized, text_field = "text")  
#corpus_quanteda
```

```
# Tokenize
```

```
tokens_quanteda <- tokens(  
  corpus_quanteda,  
  remove_punct = TRUE,  
  remove_numbers = TRUE  
)
```

```
# Remove stop words
```

```
tokens_quanteda <- tokens_remove(tokens_quanteda, stopwords("en"))
```

```
#tokens_quanteda
```

```
# Create DFM
```

```
dfm_counts <- dfm(tokens_quanteda)
```

```
dfm_counts
```

```
## Document-feature matrix of: 23 documents, 24,247 features (88.08% sparse) and 0 do
cvars.
##           features
## docs      strong imagination nourished opinion may cause men suppose edward
## A06785      3           3           1      13  84      29  33           5      7
## A06786      9           1           0      49 798      254 287          24     36
## A06788      0           1           0      11  73       50  45           5      6
## A06789      0           0           0       4  80       81  23           3      3
## A06790      2           4           0       1  11       13  38           0      0
## A06791      1           0           0       3  35       36  21           3      5
##           features
## docs      misselden
## A06785      36
## A06786       0
## A06788       0
## A06789       0
## A06790       0
## A06791       0
## [ reached max_ndoc ... 17 more documents, reached max_nfeat ... 24,237 more featur
es ]
```

```
dfm_trimmed <- dfm_trim(dfm_counts, min_termfreq = 5)
```

```
dfm_trimmed
```

```
## Document-feature matrix of: 23 documents, 7,687 features (74.43% sparse) and 0 doc
vars.
##           features
## docs      strong imagination opinion may cause men suppose edward misselden   s
## A06785      3           3      13  84      29  33           5      7          36 115
## A06786      9           1      49 798      254 287          24     36           0  11
## A06788      0           1      11  73       50  45           5      6           0   2
## A06789      0           0       4  80       81  23           3      3           0   3
## A06790      2           4       1  11       13  38           0      0           0   0
## A06791      1           0       3  35       36  21           3      5           0   0
## [ reached max_ndoc ... 17 more documents, reached max_nfeat ... 7,677 more feature
s ]
```

```
sim_r <- textstat_simil(dfm_trimmed,
                        margin = "documents",
                        method = "correlation")

r_mat <- as.matrix(sim_r)

r_mat <- round(r_mat, 3)

r_mat[1:5, 1:5]
```

```
##           A06785 A06786 A06788 A06789 A06790
## A06785   1.000   0.701   0.614   0.840   0.364
## A06786   0.701   1.000   0.726   0.712   0.506
## A06788   0.614   0.726   1.000   0.688   0.480
## A06789   0.840   0.712   0.688   1.000   0.409
## A06790   0.364   0.506   0.480   0.409   1.000
```

```
heat_df <- as.data.frame(r_mat) %>%
  rownames_to_column("doc_i") %>%
  pivot_longer(-doc_i,
               names_to = "doc_j",
               values_to = "r") %>%
  filter(doc_i != doc_j)

# Most similar pairs
most_similar <- heat_df %>%
  arrange(desc(r)) %>%
  distinct(pmin(doc_i, doc_j),
           pmax(doc_i, doc_j),
           .keep_all = TRUE) %>%
  slice_head(n = 2)

most_similar
```

```
## # A tibble: 2 × 5
##   doc_i doc_j      r `pmin(doc_i, doc_j)` `pmax(doc_i, doc_j)`
##   <chr> <chr> <dbl> <chr> <chr>
## 1 A32827 A32833 0.998 A32827 A32833
## 2 A32836 A50763 0.982 A32836 A50763
```

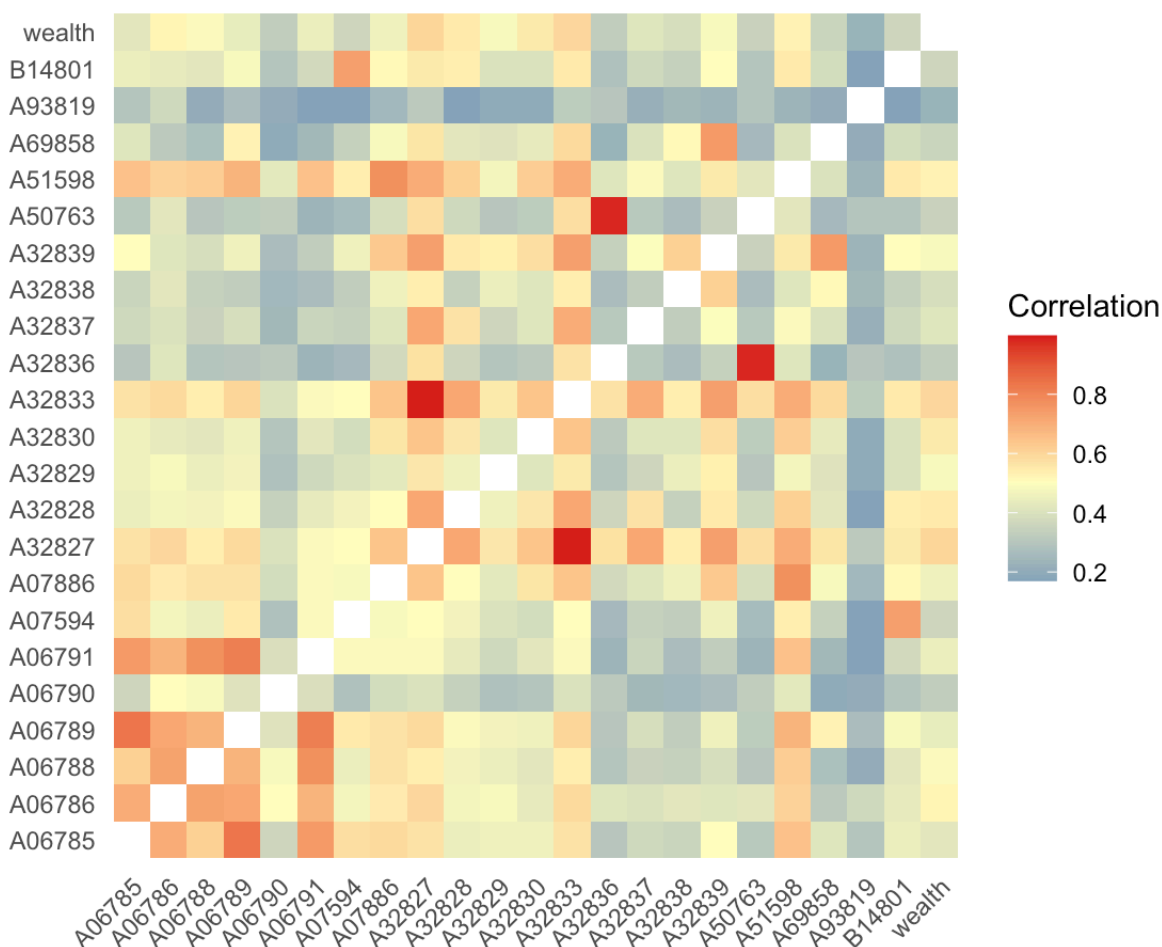
```
# Least similar pairs
least_similar <- heat_df %>%
  arrange(r) %>%
  distinct(pmin(doc_i, doc_j),
           pmax(doc_i, doc_j),
           .keep_all = TRUE) %>%
  slice_head(n = 2)
```

```
least_similar
```

```
## # A tibble: 2 × 5
##   doc_i doc_j      r `pmin(doc_i, doc_j)` `pmax(doc_i, doc_j)`
##   <chr> <chr> <dbl> <chr>                <chr>
## 1 A93819 B14801 0.17  A93819                B14801
## 2 A06791 A93819 0.174 A06791                A93819
```

```
ggplot(heat_df,
       aes(x = doc_j,
           y = doc_i,
           fill = r)) +
  geom_tile() +
  coord_fixed() +
  scale_fill_gradient2(
    low = "#2C7BB6",      # deep blue
    mid = "#FFFFBF",      # yellow
    high = "#D7191C",     # dark red
    midpoint = 0.5
  ) +
  labs(title = "Pearson Correlation Between Documents",
       x = NULL,
       y = NULL,
       fill = "Correlation") +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid = element_blank()
  )
```

Pearson Correlation Between Documents



APPROACH 3 - Syntactic complexity profile

Based on the similarity results, I selected two texts and implemented the Week 05 syntactic complexity framework. I measured sentence length, clause density, dependent clauses, coordination, and complex nominals to compare how the texts differ in grammatical structure.

```
library(udpipe)
library(dplyr)
library(stringr)
```

```
model <- udpipes_download_model(language = "english")
```

```
## Downloading udpipes model from https://raw.githubusercontent.com/jwijnffels/udpipe.m
models.ud.2.5/master/inst/udpipe-ud-2.5-191206/english-ewt-ud-2.5-191206.udpipe to /Us
ers/sejal/Documents/Text_As_Data/text_as_data/english-ewt-ud-2.5-191206.udpipe
```

```
## - This model has been trained on version 2.5 of data from https://universaldependencies.org
```

```
## - The model is distributed under the CC-BY-SA-NC license: https://creativecommons.org/licenses/by-nc-sa/4.0
```

```
## - Visit https://github.com/jwijffels/udpipe.models.ud.2.5 for model license details.
```

```
## - For a list of all models and their licenses (most models you can download with this package have either a CC-BY-SA or a CC-BY-SA-NC license) read the documentation at ?udpipe_download_model. For building your own models: visit the documentation by typing vignette('udpipe-train', package = 'udpipe')
```

```
## Downloading finished, model stored at '/Users/sejal/Documents/Text_As_Data/text_as_data/english-ewt-ud-2.5-191206.udpipe'
```

```
ud_model <- udpipe_load_model(model$file_model)
```

```
#Extract two texts
```

```
text_subset <- corpus_normalized %>%  
  filter(doc_id %in% c("A32827", "A93819"))
```

```
parsed <- udpipe_annotate(ud_model,  
                          x = text_subset$text,  
                          doc_id = text_subset$doc_id)
```

```
parsed_df <- as.data.frame(parsed)
```

```
# MLS
```

```
sentence_lengths <- parsed_df %>%  
  group_by(doc_id, sentence_id) %>%  
  summarise(words = n(), .groups = "drop") %>%  
  group_by(doc_id) %>%  
  summarise(MLS = mean(words))
```

```
# Clauses per sentence
```

```
clauses <- parsed_df %>%
  filter(upos == "VERB") %>%
  group_by(doc_id) %>%
  summarise(total_clauses = n())
```

```
sentences <- parsed_df %>%
  distinct(doc_id, sentence_id) %>%
  count(doc_id, name = "total_sentences")
```

```
cs_ratio <- clauses %>%
  left_join(sentences, by = "doc_id") %>%
  mutate(C_per_S = total_clauses / total_sentences)
```

```
colnames(parsed_df)
```

```
## [1] "doc_id"      "paragraph_id" "sentence_id"  "sentence"
## [5] "token_id"    "token"        "lemma"        "upos"
## [9] "xpos"        "feats"        "head_token_id" "dep_rel"
## [13] "deps"        "misc"
```

```
# Dependent Clauses
```

```
dependent_clauses <- parsed_df %>%
  filter(dep_rel %in% c("ccomp", "xcomp", "advcl", "acl")) %>%
  group_by(doc_id) %>%
  summarise(dep_clauses = n())
```

```
# Coordination
```

```
coordination <- parsed_df %>%
  filter(dep_rel == "conj") %>%
  group_by(doc_id) %>%
  summarise(coordination = n())
```



```
# Complex Nominals
```

```
complex_nominals <- parsed_df %>%
  filter(dep_rel %in% c("amod", "nmod", "compound")) %>%
  group_by(doc_id) %>%
  summarise(complex_nominals = n())
```

```
summary_table <- sentence_lengths %>%
  left_join(cs_ratio, by = "doc_id") %>%
  left_join(dependent_clauses, by = "doc_id") %>%
  left_join(coordination, by = "doc_id") %>%
  left_join(complex_nominals, by = "doc_id")
```

```
summary_table
```

```
## # A tibble: 2 × 8
##   doc_id  MLS total_clauses total_sentences C_per_S dep_clauses coordination
##   <chr>  <dbl>         <int>         <int>    <dbl>         <int>         <int>
## 1 A32827  63.2           4978           910     5.47           2822          3402
## 2 A93819  60.2            247            50     4.94            112           203
## # i 1 more variable: complex_nominals <int>
```

```
# Extract example sentences
```

```
example_sentences <- parsed_df %>%
  group_by(doc_id, sentence_id) %>%
  summarise(sentence = paste(token, collapse = " "),
    .groups = "drop") %>%
  group_by(doc_id) %>%
  slice_sample(n = 1)
```

```
dir.create("syntactic_results", showWarnings = FALSE)

write_csv(example_sentences,
  file.path("syntactic_results",
    "syntactic_example_sentences.csv"))
```

```
list.files("syntactic_results")
```

```
## [1] "syntactic_example_sentences.csv"
```

```

sentences_full <- parsed_df %>%
  group_by(doc_id, sentence_id) %>%
  summarise(
    sentence = paste(token, collapse = " "),
    word_count = n(),
    .groups = "drop"
  )

example_A32827 <- sentences_full %>%
  filter(doc_id == "A32827") %>%
  slice_max(word_count, n = 1)

example_A93819 <- sentences_full %>%
  filter(doc_id == "A93819") %>%
  slice_min(word_count, n = 1)

example_sentences <- bind_rows(example_A32827,
                                example_A93819)

example_sentences

```

```

## # A tibble: 2 × 4
##   doc_id sentence_id sentence                                word_count
##   <chr>         <int> <chr>                                <int>
## 1 A32827         417 i answer , those that make this objection , if ...    343
## 2 A93819         45 1647 .                                2

```

```

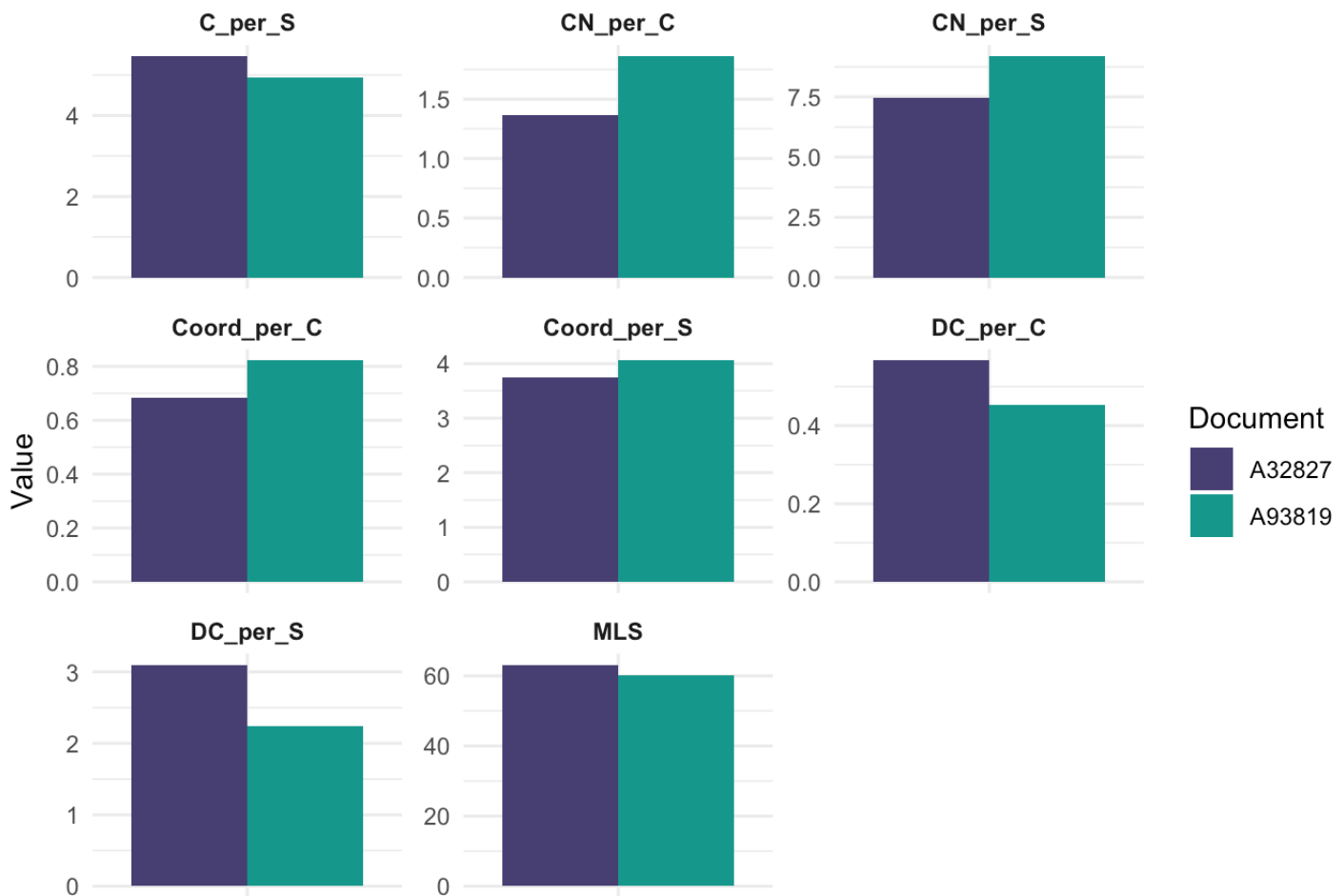
syntactic_measures <- summary_table %>%
  mutate(
    DC_per_S = dep_clauses / total_sentences,
    DC_per_C = dep_clauses / total_clauses,
    Coord_per_S = coordination / total_sentences,
    Coord_per_C = coordination / total_clauses,
    CN_per_S = complex_nominals / total_sentences,
    CN_per_C = complex_nominals / total_clauses
  ) %>%
  select(doc_id, MLS, C_per_S,
         DC_per_S, DC_per_C,
         Coord_per_S, Coord_per_C,
         CN_per_S, CN_per_C)

```

```
syntactic_long <- syntactic_measures %>%  
  pivot_longer(-doc_id,  
               names_to = "measure",  
               values_to = "value")
```

```
ggplot(syntactic_long,  
       aes(x = measure,  
           y = value,  
           fill = doc_id)) +  
  geom_col(position = "dodge") +  
  facet_wrap(~ measure, scales = "free") +  
  scale_fill_manual(values = c("#4B3F72", "#1B998B")) +  
  labs(title = "Syntactic Complexity Profile",  
       x = NULL,  
       y = "Value",  
       fill = "Document") +  
  theme_minimal() +  
  theme(axis.text.x = element_blank(),  
        strip.text = element_text(face = "bold"))
```

Syntactic Complexity Profile



Central Analytical Question Do lexical similarity patterns within the corpus correspond to shared structural (syntactic) complexity, or are vocabulary and sentence structure operating independently?

This convergence across methods suggests that variation within the corpus reflects meaningful differences in intellectual style rather than random fluctuation. Lexical clustering (Pearson), distinctiveness (TF-IDF), and syntactic density (complexity measures) all point toward internal differentiation in argumentative strategy and rhetorical construction. Importantly, Pearson correlation measures similarity in patterns of language use — not causation — but when combined with syntactic evidence, it strengthens the interpretation that the corpus contains distinct stylistic groupings within a shared intellectual tradition.